

Fall 2026 Exploratory Data Analysis with R

Lecture 7: Loops and Debugging

Brian Robertson
Federal Reserve Bank of Atlanta
2026-10-16

01

Recap & Agenda

01 Recap & Agenda

02 Loops

03 Debugging

04 Research Process & The Project

05 Recap

Quick Recap: Last Time

- **Hypothesis testing** formalizes a yes/no question about a population — form H_0/H_a , pick α , run the test, compare the p -value to α
- **Correlation** measures the sign and strength of a relationship between two variables, but never implies causation
- **Weights** change the results of both t-tests and correlation coefficients

Today: a shift from statistics to programming. We'll learn to automate repetitive tasks with **loops**, and how to find and fix the inevitable mistakes in our code — **debugging**.

Goals for Today

- Recognize when a task calls for a **loop** instead of copy-pasted code
- Write and read a `for` loop — the set, the index, and the code it runs
- Save results from a loop, with and without tracking intermediate steps
- Understand common **bug categories** and strategies for finding and fixing them
- Use `browser()` to step through code and inspect it in place
- Connect today's tools to **Project Update #5**

02

Loops

01 Recap & Agenda

02 Loops

03 Debugging

04 Research Process & The Project

05 Recap

When Do We Need a Loop?

Suppose we have test scores for five students and want to turn each one into a letter grade with an `if/else` statement.

```
test_scores <- c(85, 55, 100, 67, 91)
grade <- test_scores[1]
# ...run the if/else code...
```

That works for one student. What if we had 100 students? Re-running the same code by hand, once per student, isn't practical.

Loops Automate Repetitive Tasks

- Preparing data for analysis often means repeating the same calculation many times
- Doing this by hand is:
 - Time-consuming
 - Error-prone — easy to mistype an index or skip a case
 - Bad for readability — the same five lines, copy-pasted 100 times

A **loop** runs the same block of code once per element of a set, so we write the logic once and let R repeat it.

Components of a **for** Loop

Every **for** loop has three parts:

1. A **set** — the numbers or elements the loop iterates over
2. An **index** — a variable that refers to the current element of the set
3. The **code** to run once per element

Anatomy of a **for** Loop

```
for (index in set) {  
  # code to run, once per value of index  
}
```

- The parentheses hold the set and the index; the braces hold the code — the same shape as an **if/else** statement
- The name of the index variable is up to you, but it must be used **consistently** — R has no idea what **score** means if you looped over **i**

A Simple Example

Let's square every number from 2 to 6 and print the result.

```
example_vector <- c(2:6)
example_vector
```

```
[1] 2 3 4 5 6
```

One option: do it one value at a time.

```
print(example_vector[1]^2)
print(example_vector[2]^2)
print(example_vector[3]^2)
print(example_vector[4]^2)
print(example_vector[5]^2)
```

To turn this into a loop: what's the set? It's 2:6.

A Simple Example, as a Loop

```
for (i in 2:6) {  
  print(i^2)  
}
```

```
[1] 4  
[1] 9  
[1] 16  
[1] 25  
[1] 36
```

- The index variable is `i`
- The code that runs once per element is `print(i^2)`

Looping Over a Vector Directly

`for` loops can iterate over a vector or list directly, not just a numeric range.

```
for (i in example_vector) {  
  print(paste("The index is", i))  
  print(paste("Which squared is:", i^2))  
}
```

```
[1] "The index is 2"  
[1] "Which squared is: 4"  
[1] "The index is 3"  
[1] "Which squared is: 9"  
[1] "The index is 4"  
[1] "Which squared is: 16"  
[1] "The index is 5"  
[1] "Which squared is: 25"
```

```
[1] "The index is 6"  
[1] "Which squared is: 36"
```

Looping Over Our Test Scores

```
for (i in test_scores) {  
  print(paste("Student's test score is", i))  
}
```

```
[1] "Student's test score is 85"  
[1] "Student's test score is 55"  
[1] "Student's test score is 100"  
[1] "Student's test score is 67"  
[1] "Student's test score is 91"
```

What would happen if we replaced `i` with `score` everywhere in the loop?

Naming the Index Consistently

```
for (score in test_scores) {  
    print(paste("Student's test score is", score))  
}
```

```
[1] "Student's test score is 85"  
[1] "Student's test score is 55"  
[1] "Student's test score is 100"  
[1] "Student's test score is 67"  
[1] "Student's test score is 91"
```

- Same result — the index name doesn't matter, as long as it's used consistently

What happens if the names *don't* match?

```
for (score in test_scores) {  
  print(paste("Student's test score is", i))  
}
```

Error:
! object 'i' not found

A Note on Sets and Indices

We can also use the set to index into a vector from inside the loop:

```
for (i in 1:10) {  
  print(paste0("Student ", i, "'s test score is ", test_scores[i]))  
}
```

- Here the set `1:10` doesn't contain the scores themselves — it's used to look up each score by position

Being Careful About Your Set

`test_scores` only has 5 elements. What happens if we loop from 1 to 12?

```
for (i in 1:12) {  
  print(test_scores[i])  
}
```

```
[1] 85  
[1] 55  
[1] 100  
[1] 67  
[1] 91  
[1] NA  
[1] NA  
[1] NA  
[1] NA
```

```
[1] NA  
[1] NA  
[1] NA
```

- The set is longer than the vector we're indexing into — once we run out of scores, R just returns **NA**

Avoiding Hard-Coded Sets

Writing the set by hand (`1:12`, `1:5`, ...) invites exactly this kind of mistake. `length()` and `seq_along()` size the set to the data automatically:

```
for (i in seq_along(test_scores)) {  
  print(test_scores[i])  
}
```

```
[1] 85  
[1] 55  
[1] 100  
[1] 67  
[1] 91
```

- `seq_along(test_scores)` is `1:length(test_scores)` — but it updates automatically if `test_scores` changes length, so prefer it over hard-coding the range

Saving Results From a Loop

Sometimes we only care about the final answer; other times we want to track every intermediate step. Let's compute a **cumulative sum** — a running total — as an example.

- Use an index for each element of the original vector
- To save results, first **initialize** a vector to hold them, outside the loop
- Compute the next value using a helper variable, then append it to the result vector

Without Saving Intermediate Results

```
num_vec <- 5:10

for (current_num in seq_along(num_vec)) {
  current_cumul_sum <- sum(num_vec[1:current_num])
}

current_cumul_sum
```

```
[1] 45
```

- No `print()` inside the loop, so we see nothing while it runs — but afterward, `current_cumul_sum` holds the *last* value computed, not the full history

Tracking Progress Inside a Loop

Adding a `print()` statement lets us watch the loop work, which is useful when iterating over a lot of elements.

```
num_vec <- 5:10

for (current_num in seq_along(num_vec)) {
  current_cumul_sum <- sum(num_vec[1:current_num])
  print(current_cumul_sum)
}
```

```
[1] 5
[1] 11
[1] 18
[1] 26
```

```
[1] 35  
[1] 45
```

- Printing shows us every step, but still doesn't save them — we're left with only the final `current_cumul_sum`

Saving Every Intermediate Result

To keep the whole history, initialize an empty object **before** the loop, then append to it on every iteration.

```
num_vec <- 5:10
cumul_sum <- NULL

for (current_num in seq_along(num_vec)) {
  current_cumul_sum <- sum(num_vec[1:current_num])
  cumul_sum <- c(cumul_sum, current_cumul_sum)
}

cumul_sum
```

```
[1]  5 11 18 26 35 45
```

- `cumul_sum` now holds every intermediate value, in order

if/else Combined With a Loop

Let's revisit letter grades — now we can grade every student in one pass, and save every result.

```
test_scores <- c(85, 55, 100, 67, 73, 92, 94, 99, 87, 89.3)
letter_grades <- NULL

for (score in test_scores) {
  if (score >= 90) {
    letter_grades <- c(letter_grades, "A")
  } else if (score >= 80) {
    letter_grades <- c(letter_grades, "B")
  } else if (score >= 70) {
    letter_grades <- c(letter_grades, "C")
  } else if (score >= 60) {
    letter_grades <- c(letter_grades, "D")
  } else {
    letter_grades <- c(letter_grades, "F")
  }
}
```

```
}  
}  
  
letter_grades
```

```
[1] "B" "F" "A" "D" "C" "A" "A" "A" "B" "B"
```

Breakout: Budgeting With Loops

You've been tracking how much you spend each month.

```
monthly_spending <- c(703, 200, 474, 576, 636, 369,  
                      549, 553, 652, 435, 724, 910)
```

1. Use a `for` loop to compute your **cumulative average** spending each month. Save every intermediate result in a vector.
2. Combine a loop and an `if/else` statement to flag which months your cumulative average was over your \$520 limit. Save the result in a vector.
3. **Bonus:** plot the cumulative average spent by month as a line chart.

Solution: Question 1

```
cumul_avg_spent <- NULL

for (i in seq_along(monthly_spending)) {
  current_avg <- sum(monthly_spending[1:i]) / i
  cumul_avg_spent <- c(cumul_avg_spent, current_avg)
}

cumul_avg_spent
```

```
[1] 703.0000 451.5000 459.0000 488.2500 517.8000 493.0000 501.0000 507.5000
[9] 523.5556 514.7000 533.7273 565.0833
```

Solution: Question 2

```
my_budget <- NULL

for (avg in cumul_avg_spent) {
  if (avg > 520) {
    my_budget <- c(my_budget, "OVER")
  } else {
    my_budget <- c(my_budget, "UNDER")
  }
}

my_budget
```

```
[1] "OVER" "UNDER" "UNDER" "UNDER" "UNDER" "UNDER" "UNDER" "UNDER" "OVER"
[10] "UNDER" "OVER" "OVER"
```

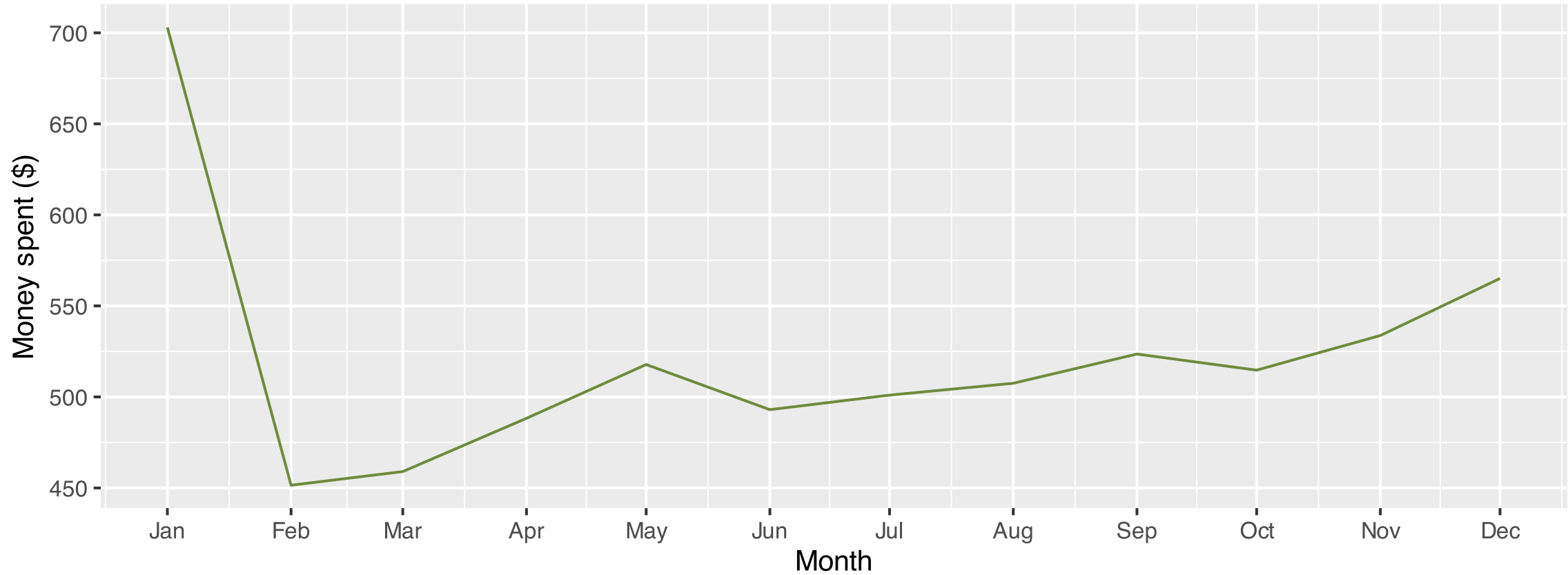
Solution: Bonus

```
month_seq <- seq.Date(as.Date("2026-01-01"), as.Date("2026-12-01"), by = "month")

budget_df <- tibble(month_start_date = month_seq,
                    cumul_avg = cumul_avg_spent)

ggplot(budget_df, aes(x = month_start_date, y = cumul_avg)) +
  geom_line(color = "darkolivegreen4") +
  labs(title = "Monthly Budget Tracking", x = "Month", y = "Money spent ($)") +
  scale_x_date(date_breaks = "1 month", date_labels = "%b")
```

Monthly Budget Tracking



03

Debugging

- 01 Recap & Agenda
- 02 Loops
- 03 Debugging**
- 04 Research Process & The Project
- 05 Recap

What Is a Bug?

- Originally, computers used vacuum tubes, and bugs were literally insects shorting them out
- Today, any problem in code that leads to unintended output is a “bug” — from `a < - 1` instead of `a < - 1`, to a subtle logic error
- Bugs are common, and nothing to be ashamed of — everyone writes buggy code
- Industry estimates put the average at 15–50 defects per 1,000 lines of code (McConnell, 2004)
- Companies run **bug bounty programs**, paying people to find bugs before the public does

Finding Bugs

When your code isn't behaving as expected, you need to:

1. Figure out *why* the wrong output is being produced
2. Find *where* in the code that happened

Two approaches:

- **Manually** — read the code, trace the logic, check intermediate values
- **Using R's built-in tools** — `browser()`, error messages, and warnings

Bug Categories

A non-exhaustive list:

- **Syntax issues** — incorrect syntax, or valid syntax that doesn't do what you meant
- **Violated assumptions** — missing dependencies, undefined variables, variables that don't contain what you think
- **Logic problems** — an algorithm that's implemented incorrectly, or corner cases you didn't account for
- **Misc. silly mistakes** — forgetting to remove a debugging step before sharing code

Knowing the category helps you know where to look.

Incorrect Syntax

We meant to count how many characters are tall vs. not — but we reused `count` as both the new column name and a function call.

```
starwars %>%  
  mutate(tall = height > 100) %>%  
  group_by(tall) %>%  
  summarise(count = count(tall))
```

```
Error in `summarise()`:  
i In argument: `count = count(tall)`.  
i In group 1: `tall = FALSE`.  
Caused by error in `UseMethod()`:  
! no applicable method for 'count' applied to an object of class "logical"
```

Valid Syntax, Unintended Outcome

A single `=` inside the parentheses isn't a comparison — it's an assignment, so every row gets `tall = 100`.

```
starwars %>%  
  mutate(tall = (height = 100)) %>%  
  group_by(tall) %>%  
  summarise(count = n())
```

```
# A tibble: 1 × 2  
  tall count  
  <dbl> <int>  
1   100     87
```

Fix: use `>` to compare, not `=` to assign

```
starwars %>%  
  mutate(tall = height > 100) %>%  
  group_by(tall) %>%  
  summarise(count = n())
```

```
# A tibble: 3 × 2  
  tall    count  
  <lgl> <int>  
1 FALSE     7  
2 TRUE     74  
3 NA        6
```

Missing Dependencies

We're missing a function, a package, or a file that our code depends on.

```
starwars %>% tabyl(hair_color, gender)
```

```
Error in `tabyl()`:  
! could not find function "tabyl"
```

Fix: load the package that defines it

```
library(janitor)  
starwars %>% tabyl(hair_color, gender) %>% head(4)
```

```
hair_color feminine masculine NA_  
auburn          1           0    0
```

auburn, grey	0	1	0
auburn, white	0	1	0
black	3	9	1

Variable Contents Issues

We may believe a column contains what we think it does — or we may have its name wrong.

```
starwars %>%  
  mutate(stewjonite = (home == "Stewjon")) %>%  
  pull(stewjonite)
```

```
Error in `mutate()`:  
i In argument: `stewjonite = (home == "Stewjon")`.  
Caused by error:  
! object 'home' not found
```

Fix: the column is called `homeworld`, not `home`

```
starwars %>%  
  mutate(stewjonite = (homeworld == "Stewjon")) %>%
```

```
pull(stewjonite) %>%  
head()
```

```
[1] FALSE FALSE FALSE FALSE FALSE FALSE
```

Logic Problems

- Maybe your algorithm isn't correctly implemented — trace your variables at each step and check they're what you expect
- Maybe you haven't accounted for a **corner case** — a non-standard input your code doesn't handle

Algorithm Not Correctly Implemented

```
grade <- 90
if (grade >= 90 & grade >= 100) {
  letter_grade <- "A"
} else {
  letter_grade <- NA
}
letter_grade
```

```
[1] NA
```

- We wanted “A if 90 or above,” but wrote “A if 90 or above **and** 100 or above” — a 90 should be an A, not NA

Forgetting to Remove a Debugging Step

```
complicated_math <- function(a, b) {  
  a <- a + b  
  answer <- a + b + 5  
  print(a)  
}  
  
answer <- complicated_math(5, 6)
```

```
[1] 11
```

```
answer
```

```
[1] 11
```

- We wanted 22 (`a + b + 5` with the updated `a`), but got 11
- A function returns its *last evaluated expression* — here that's `print(a)`, which returns `a` invisibly. The real `answer` variable never leaves the function; a leftover debugging `print()` silently papered over a missing `return(answer)`

Debugging Is Hard

- Code runs linearly — every line depends on everything before it
- So debugging is like peeling an onion and reconstructing it, layer by layer
- Sometimes the mistake is so small you can't imagine it's there

Strategies

- Keep a checklist of errors and corner cases you've hit before
- Comment your assumptions as you write code — your future self will thank you
- Google error messages you don't recognize
- Run code one chunk at a time until the error shows up, then work backwards from there
- Add print statements, or pause execution and inspect
- Clear your environment and re-run from a clean session before sharing code — “works on my machine” often means a stray variable is doing the work

browser()

`browser()` pauses execution wherever you place it, and drops you into a console **inside that point in your code**.

```
divide <- function(numerator, denominator) {  
  if (denominator == 0) {  
    browser()  
  }  
  numerator / denominator  
}
```

- When R hits `browser()`, you get a `Browse[1]>` prompt — inspect any variable in scope, or run code to see how it behaves
- Type `c` and press enter to continue; remove `browser()` before you share or submit your code
- See `?browser`, and <https://adv-r.hadley.nz/debugging.html> for more tools

Breakout: Debugging With `print()` and `browser()`

- Open `lecture07_inclass_exercise.R`
- Work through the broken functions in the debugging section
- For each one: read the error, form a hypothesis about what's wrong, then confirm it with `print()` statements or `browser()`
- 10 minutes, then a 5-minute break

04

Research Process & The Project

- 01 Recap & Agenda
- 02 Loops
- 03 Debugging
- 04 Research Process & The Project**
- 05 Recap

Research Process: Project Update #5

Project Update #5 asks you to:

- **Iterate** on Project Update #4 — revise it based on the feedback you received
- **Refine your interpretation** — what did you originally expect, and how do your findings compare?
- Run a **t-test** on a relationship in your own data

Either way, you'll also follow up with your adviser and post an update for your classmates.

This Week's Timeline

- **HW #7 — due October 22.** Today's material: loops and debugging.
- **Project Update #5 — due October 24.** Iterating on feedback, refined interpretation, and a t-test on your own data.

We now have some time in class for you to work on your project — flag your adviser or a TA if you get stuck.

05

Recap

- 01 Recap & Agenda
- 02 Loops
- 03 Debugging
- 04 Research Process & The Project
- 05 Recap**

Recap

- **Loops** automate repetitive tasks — a `for` loop needs a set, an index, and code to run per element
- `seq_along()` sizes a loop to your data, instead of a hard-coded range that can run off the end of a vector
- Save intermediate results by initializing a vector before the loop and appending to it inside the loop
- **Bugs** fall into a few recognizable categories — syntax, violated assumptions, logic, and silly mistakes
- `browser()` lets you pause execution and inspect your code from the inside

Functions Introduced Today

- `seq_along()` — generates an index sequence sized to a vector
- `browser()` — pauses execution for interactive inspection
- `janitor::tabyl()` — quick cross-tabulations

Documentation & Further Reading

- `?for`, `?seq_along` — loop syntax and safe indexing
- `?browser` — interactive debugging
- <https://adv-r.hadley.nz/debugging.html> — a deeper tour of R's debugging tools
- `?janitor::tabyl` — quick cross-tabulations

Coming up: advanced plots and an introduction to regression.